

CAPITULO ESTUDIANTEL ACM - ICPC UMSA

3

CAMPAMENTO DE PROGRAMACIÓN COMPETITIVA

ENERO 2019





3. Estructuras de datos



Kevin Arturo Navia Paiva
Oscar Gauss Carvajal Yucra

Carrera de Informatica

Facultad de Ciencias Puras y Naturales



Estructuras de datos



- 1 Lazy propagation
- 2 Persistencia
- 3 Treap
Definición
- 4 Operaciones
Operaciones treap
Operaciones binary search tree
- 5 Treap implícito
Definición



1

Lazy propagation



UMSA
la mejor

Lazy propagation



En ocasiones puede ser necesaria una estructura que permita realizar modificaciones sobre muchos elementos a la vez, por ejemplo actualizar el rango $[a, b]$ fijándolo en un cierto valor o sumándole a cada elemento una cantidad dada:

- Los segment trees soportan estas operaciones en $O(N \log N)$, pero queremos algo más eficiente.
- El poder del segment tree radica en que todo segmento puede ser representado mediante la unión de $O(\log N)$ segmentos disjuntos.
- Podemos alcanzar un tiempo de ejecución $O(\log N)$ si ampliamos la estructura para "marcar" segmentos sobre los que queremos realizar una operación (a ser efectuada sólo en caso de ser necesario).



Lazy propagation



En general requerimos por cada nodo/segmento:

- Los parámetros de la operación que debemos aplicar(incluyendo posiblemente un flag para indicar si hay una operación pendiente que no hemos efectuado aún);
- Una forma de calcular el valor asociado a todo el segmento sin realizar la operación pendiente;
- Una forma de “partir” operaciones en dos partes, para “transferirlas” a los hijos de un nodo si este no está totalmente contenido en el segmento que nos interesa;

La estructura funciona como un segment tree usual pero que “empuja” las operaciones pendientes hacia las hojas a medida que realiza otras operaciones. El tiempo de ejecución por operación es $O(\log N)$ amortizado.



2

Persistencia



UMSA
la mejor



Nos interesa una estructura de datos con “ventana al pasado”:

- Queremos realizar K operaciones de modificación de los datos;
- Las consultas deben responderse con la estructura tal como era justo después de la k -ésima modificación, pero pueden realizarse después de haber realizado más operaciones;
- Guardar K copias completas de la estructura de datos es demasiado costoso.

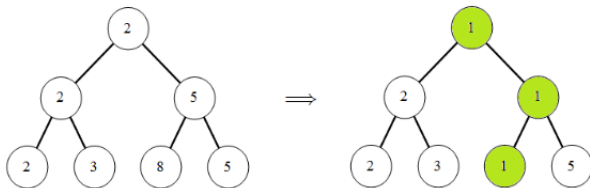
Observación: Al modificar una estructura, la mayor parte de la misma no sufre cambios, de modo que puede reutilizarse.



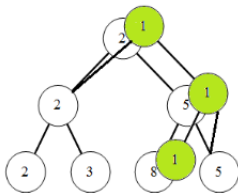
Persistencia



Usual



Persistente





- Mantenemos una lista de raíces R_k para $k = 1, \dots, K$
- Para responder consultas utilizamos una función similar a la del segment tree usual empezando en la raíz R_k deseada y cambiando $LEFT(n)$ y $RIGHT(n)$ por punteros a (o identificadores de) los hijos de cada nodo.
- Para modificar los datos generamos nuevos nodos en lugar de modificar los existentes en la versión de referencia;
- Cada nuevo nodo tiene un "reflejo" en la versión de referencia que nos indica cuál es el nodo correspondiente al hijo que no sufre modificaciones;
- Por cada modificación generamos solamente $O(\log N)$.



Implementación de la Inserción)



```
void insert(int cur, int s, int e, int ref, int p,
            int v) {
    int left, right, m;
    if (e - s > 1) {
        m = (s + e) / 2;
        if (p < m) {
            left = tree[cur].l = NODES++;
            right = tree[cur].r = tree[ref].r;
            insert(left, s, m, tree[ref].l, p, v);
        } else {
            left = tree[cur].l = tree[ref].l;
            right = tree[cur].r = NODES++;
            insert(right, m, e, tree[ref].r, p, v);
        }
        tree[cur].v = oper(tree[left].v, tree[right].v);
    } else tree[cur].v = v;
}
```



3

Treap



UMSA
la mejor

Algunas ideas



Un Treap es un arbol binario de busqueda autobalanceable.
El Treap es una estructura que puede ser usada para representar otro tipo de estrcuturas(Segment Tree, Fenwick Tree, Interval Tree).

TREAP = TREE + HEAP.



Binary Search Tree



Un BST es un árbol que cumple las siguientes propiedades:

- Cada nodo tiene una clave única que determina el orden total.
- La clave de cualquier nodo es mayor a todas las claves del subárbol izquierdo y es menos a todas las claves del subárbol derecho

BST esta representado en C++ por **set**



Binary Search Tree



Binary Heap



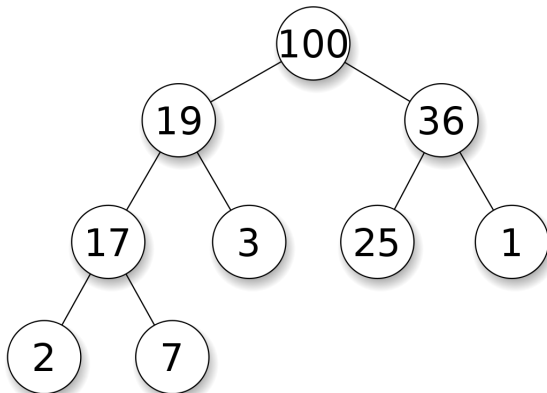
Un Heap es un arbol que cumple la siguiente propiedad.

- La prioridad de un nodo siempre es mayor que la prioridad de sus nodos hijos.

Heap esta representado en C++ por ***priority_queue***

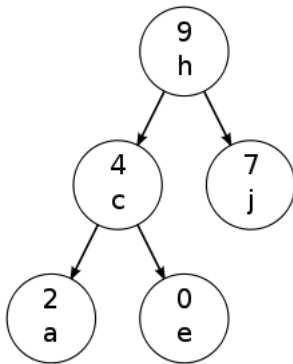


Binary Heap



Definición

Treap es un árbol binario, cada nodo contiene dos valores: una prioridad y una clave (H, B).





Treap cumple las siguientes propiedades:

- $H_{izq}(i) < H_i > H_{der}(i)$ y
- $B_{izq}(i) < B_i < B_{der}(i)$
- Las prioridades se asignan de manera aleatoria.
- La anterior propiedad garantiza que la altura del árbol sea $O(\lg(n))$ como valor mas probable.



Implementación



```
struct node{
    node *l, *r;
    int val, prior;
};

typedef node* pnode;

pnode init(int val){
    pnode ret;
    ret->val = val; ret->prior=rand(); ret->l = ret->r =
    return ret;
}
```



4

Operaciones



UMSA
la mejor

Operaciones treap



El treap soporta dos tipos de operaciones ambas en $O(\lg(n))$:

- **split(T, X)** Divide el treap T en dos treaps L y R tal que L contiene todos los nodos con $B_k \leq X$ y el treap R contiene todos los nodos con $B_k > X$. El treap T es destruido durante la operación.
- **merge(L, R)** Une dos treaps L y R en un solo treap T . Los treaps L y R se destruyen después de la operación. Nótese que las claves de los nodos del treap L son menores que las claves de los nodos del treap R .



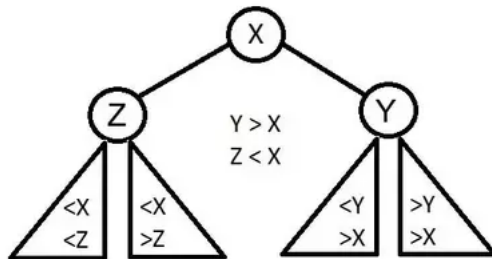
Split(Implementacion)



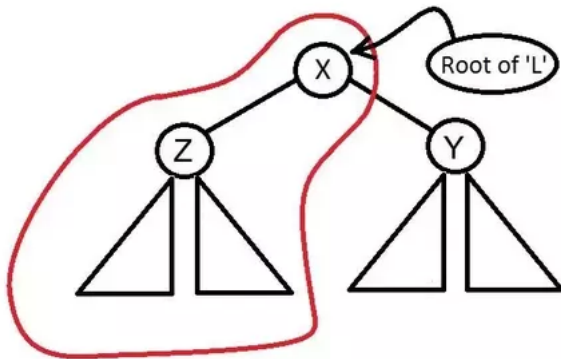
```
void split(pnode t, pnode &l, pnode &r, int key){  
    if(!t) l = r = NULL;  
    else if(t->val <= key)  
        split(t->r, t->r, r, key), l = t;  
    else  
        split(t->l, l, t->l, key), r = t;  
}
```



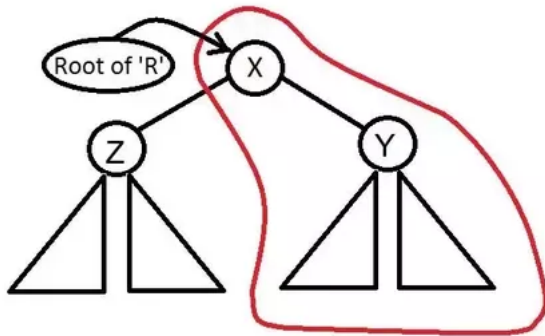
Split



Split



Split



Merge(Implementacion)



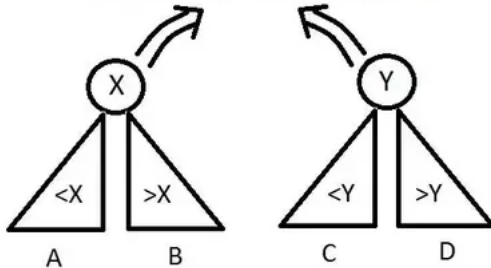
```
void merge(pnode &t,pnode l,pnode r){  
    if(!l || !r) t = l ? l : r;  
    else if(l->prior > r->prior)  
        merge(l->r, l->r, r), t = l;  
    else  
        merge(r->l, l, r->l), t = r;  
}
```



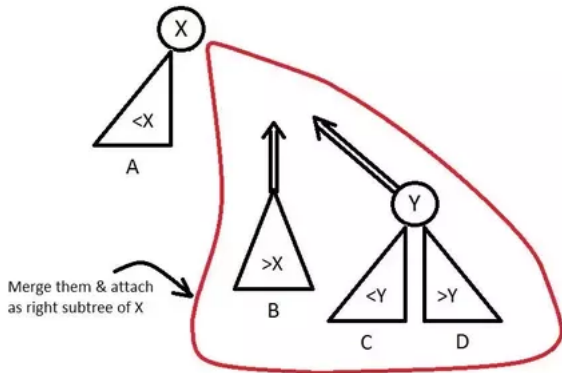
Merge



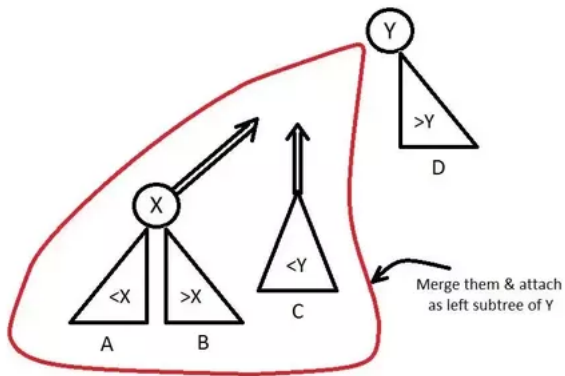
We need to Merge them into a single tree



Merge



Merge



Operaciones bst



El treap soporta tres operaciones que corresponden al bst:

- **find(X)** Operación básica de un bst devuelve el nodo donde se encuentra el valor buscado, en caso de no existir retorna nulo.
- **insert(X)** Inserta una clave X en el treap, previamente selecciona el azar un valor Y que corresponde a la prioridad; forma un par(X, Y). Encontrar el nodo para el cual $Y > \text{prior}$. Hacer split en el subarbol y luego utilizar un merge para reensamblar el treap.
- **erase(x)** Encontrar el nodo que contenga la clave X, y realizar un merge entre los subárboles izquierdo y derecho para volver a reinsertar el resto del treap.



Find (Implementación)



```
pnode find(pnode &t, int X) {  
    if(!t) return NULL;  
    if(t->val == X) {  
        return t;  
    }  
    if(X < t->val)  
        return find(t->l, X);  
    else  
        return find(t->r, X);  
}
```



Insert (Implementación)



```
void insert(pnode &t, pnode it){  
    if(!t) t = it;  
    else if(it->prior > t->prior)  
        split(t, it->l, it->r, it->val) , t = it;  
    else  
        insert(t->val <= it->val ? t->r : t->l, it);  
}
```



Erase (Implementación)



```
void erase(pnode &t, int key){  
    if(!t) return;  
    else if(t->val == key) {  
        pnode temp = t; merge(t, t->l, t->r);  
    } else  
        erase(t->val < key? t->r: t->l, key);  
}
```



Información adicional



Cada nodo puede albergar informacion adicional:

- Un valor correspondiente al nodo. Asi como un **map**.
- El tamaño del subárbol izquierdo, lo cual nos permite hallar el k-ésimo elemento en el árbol.
- La suma de todos los valores de del subárbol izquierdo, entre otros...



Ejemplo



5

Treap implícito



UMSA
la mejor

Definición



El treap implícito es una modificación del treap, donde el treap pasa a representar un arreglo. Mantiene las propiedades del treap y añade algunas:

- Como el treap normal, soporta las operaciones split y merge.
- Las claves de los nodos ahora son los índices del arreglo con lo cual garantizamos que el arreglo nunca se deforma.



Qué se puede hacer con un treap implícito?



- Puede guardar informacion en rangos(subárbol izquierdo y derecho).
- Puede representar segment trees, fenwick trees.
- Realizar operaciones lazy.
- Realizar en segmentos del arreglo.

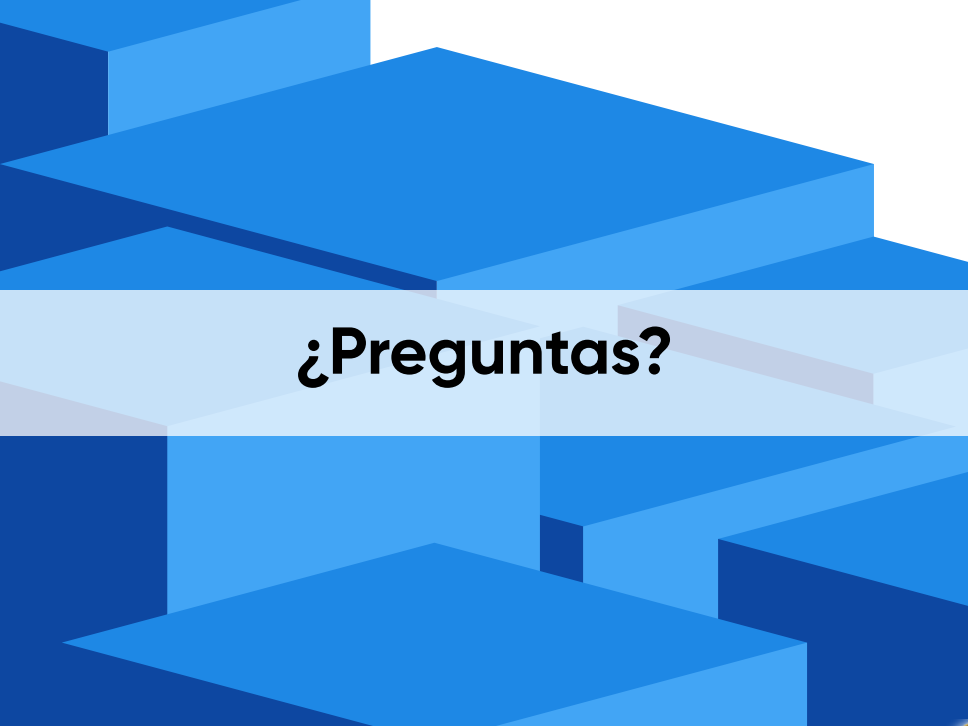


Ejemplo



Ejercicio



The background consists of several overlapping, three-dimensional rectangular blocks in various shades of blue (dark blue, medium blue, and light blue) and white. These blocks are arranged in a way that creates a sense of depth and perspective, with some blocks appearing to be in front of others. The overall composition is clean and modern.

¿Preguntas?

The background is an abstract composition of various geometric shapes, primarily squares and rectangles, in different shades of blue (dark blue, medium blue, light blue) and white. These shapes are arranged in a way that creates a sense of depth and perspective, resembling a stylized architectural structure or a series of steps. The lighting is soft, with subtle gradients and shadows that enhance the three-dimensional effect.

Gracias!